

```

import numpy as np
import pandas as pd

# -----
# Distance Metrics
# -----

def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

def manhattan_distance(a, b):
    return np.sum(np.abs(a - b))

# -----
# k-NN Regressor From Scratch
# -----

class KNNRegressor:

    def __init__(self, k=5, metric="euclidean"):
        self.k = k
        self.metric = metric

    def fit(self, X, y):
        self.X_train = np.asarray(X, dtype=float)
        self.y_train = np.asarray(y, dtype=float)
        return self

    def calculate_distance(self, a, b):

        if self.metric == "euclidean":
            return euclidean_distance(a, b)

        elif self.metric == "manhattan":
            return manhattan_distance(a, b)

        else:
            raise ValueError("Unknown distance metric")

    def predict(self, X):

        X = np.asarray(X, dtype=float)
        predictions = []

        for sample in X:

            distances = []

            for i, train_sample in enumerate(self.X_train):

                distance = self.calculate_distance(
                    sample,
                    train_sample
                )

                distances.append((distance, self.y_train[i]))

```

```

        # Sort according to distance
        distances.sort(key=lambda x: x[0])

        # Select k nearest neighbours
        nearest = distances[:self.k]

        # Average their target values
        prediction = np.mean(
            [value for _, value in nearest]
        )

        predictions.append(prediction)

    return np.array(predictions)

# -----
# Mean Absolute Error
# -----

def mean_absolute_error(y_true, y_pred):

    return np.mean(
        np.abs(np.asarray(y_true) - np.asarray(y_pred))
    )

# -----
# Example Dataset
# -----

data = pd.DataFrame({
    "rainfall": [500, 600, 700, 800, 900, 1000,
                1100, 1200, 1300, 1400],

    "temperature": [32, 31, 30, 29, 28, 27,
                   26, 27, 28, 29],

    "irrigation": [100, 120, 140, 160, 180, 200,
                  220, 240, 260, 280],

    "yield": [2.1, 2.4, 2.8, 3.0, 3.3, 3.7,
              4.0, 4.2, 4.5, 4.7]
})

# -----
# Features and Target
# -----

X = data[
    ["rainfall", "temperature", "irrigation"]
].values

y = data["yield"].values

# -----
# Train / Validation Split
# -----

split = int(0.8 * len(X))

```

```

X_train = X[:split]
X_val = X[split:]

y_train = y[:split]
y_val = y[split:]

# -----
# Feature Scaling
# -----

mean = X_train.mean(axis=0)
std = X_train.std(axis=0)

std[std == 0] = 1

X_train_scaled = (X_train - mean) / std
X_val_scaled = (X_val - mean) / std

# -----
# Manual Validation Curve
# -----

k_values = [1, 2, 3, 4, 5]

results = []

for metric in ["euclidean", "manhattan"]:

    for k in k_values:

        model = KNNRegressor(
            k=k,
            metric=metric
        )

        model.fit(
            X_train_scaled,
            y_train
        )

        predictions = model.predict(
            X_val_scaled
        )

        error = mean_absolute_error(
            y_val,
            predictions
        )

        results.append({
            "Metric": metric,
            "k": k,
            "Validation_MAE": error
        })

results_df = pd.DataFrame(results)

print("\nValidation Results:")

```

```

print(results_df)

# -----
# Best k
# -----

best_row = results_df.loc[
    results_df["Validation_MAE"].idxmin()
]

print("\nBest Configuration:")
print(best_row)

# -----
# Final Prediction
# -----

best_k = int(best_row["k"])
best_metric = best_row["Metric"]

final_model = KNNRegressor(
    k=best_k,
    metric=best_metric
)

final_model.fit(
    X_train_scaled,
    y_train
)

predictions = final_model.predict(
    X_val_scaled
)

print("\nActual Values:")
print(y_val)

print("\nPredicted Values:")
print(predictions)

print("\nFinal MAE:",
      mean_absolute_error(y_val, predictions))

```

Validation Results:

	Metric	k	Validation_MAE
0	euclidean	1	0.400000
1	euclidean	2	0.500000
2	euclidean	3	0.633333
3	euclidean	4	0.800000
4	euclidean	5	0.960000
5	manhattan	1	0.400000
6	manhattan	2	0.500000
7	manhattan	3	0.633333
8	manhattan	4	0.800000
9	manhattan	5	0.960000

Best Configuration:

Metric	euclidean
k	1
Validation_MAE	0.4
Name: 0, dtype: object	

Actual Values:

[4.5 4.7]

Predicted Values:

[4.2 4.2]

Final MAE: 0.3999999999999999

```
import numpy as np
import pandas as pd

# -----
# Mean Absolute Error
# -----

def mean_absolute_error(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred))

# -----
# Gaussian Kernel
# -----

def gaussian_kernel(distance, tau):
    return np.exp(
        -(distance ** 2) / (2 * tau ** 2)
    )

# -----
# Locally Weighted Regression
# -----

class LocallyWeightedRegression:

    def __init__(self, tau=1.0):
        self.tau = tau

    def fit(self, X, y):
        self.X_train = np.asarray(X, dtype=float)
        self.y_train = np.asarray(y, dtype=float)

    def predict_point(self, x):

        x = np.asarray(x, dtype=float)

        # Add intercept
        X = np.column_stack(
            (np.ones(len(self.X_train)),
             self.X_train)
        )

        x_with_intercept = np.insert(x, 0, 1)

        # Calculate distances
        distances = np.sqrt(
            np.sum(
                (self.X_train - x) ** 2,
                axis=1
            )
        )
```

```

    )

    # Calculate weights
    weights = np.array([
        gaussian_kernel(d, self.tau)
        for d in distances
    ])

    # Weight matrix
    W = np.diag(weights)

    # Calculate theta
    XT_W = X.T @ W

    A = XT_W @ X
    B = XT_W @ self.y_train

    # Small regularisation for numerical stability
    A = A + 1e-8 * np.eye(A.shape[0])

    theta = np.linalg.solve(A, B)

    # Prediction
    prediction = x_with_intercept @ theta

    return prediction

def predict(self, X):

    predictions = []

    for x in X:
        predictions.append(
            self.predict_point(x)
        )

    return np.array(predictions)

# -----
# Example Dataset
# -----

data = pd.DataFrame({
    "rainfall": [
        500, 600, 700, 800, 900,
        1000, 1100, 1200, 1300, 1400
    ],
    "temperature": [
        32, 31, 30, 29, 28,
        27, 26, 27, 28, 29
    ],
    "irrigation": [
        100, 120, 140, 160, 180,
        200, 220, 240, 260, 280
    ],
    "yield": [
        2.1, 2.4, 2.8, 3.0, 3.3,
        3.7, 4.0, 4.2, 4.5, 4.7
    ]
})

```

```

    ]
})

# -----
# Features and Target
# -----

X = data[
    ["rainfall", "temperature", "irrigation"]
].values

y = data["yield"].values

# -----
# Train / Test Split
# -----

split = int(0.8 * len(X))

X_train = X[:split]
X_test = X[split:]

y_train = y[:split]
y_test = y[split:]

# -----
# Feature Scaling
# -----

mean = X_train.mean(axis=0)
std = X_train.std(axis=0)

std[std == 0] = 1

X_train_scaled = (
    X_train - mean
) / std

X_test_scaled = (
    X_test - mean
) / std

# -----
# Test Different Bandwidths
# -----

tau_values = [
    0.1,
    0.3,
    0.5,
    1.0,
    2.0
]

results = []

for tau in tau_values:

```

```

        model = LocallyWeightedRegression(
            tau=tau
        )

        model.fit(
            X_train_scaled,
            y_train
        )

        predictions = model.predict(
            X_test_scaled
        )

        error = mean_absolute_error(
            y_test,
            predictions
        )

        results.append({
            "Tau": tau,
            "MAE": error
        })

results_df = pd.DataFrame(results)

print("\nLWR Results:")
print(results_df)

# -----
# Best Tau
# -----

best = results_df.loc[
    results_df["MAE"].idxmin()
]

print("\nBest Bandwidth:")
print(best)

# -----
# Final Model
# -----

best_tau = best["Tau"]

final_model = LocallyWeightedRegression(
    tau=best_tau
)

final_model.fit(
    X_train_scaled,
    y_train
)

final_predictions = final_model.predict(
    X_test_scaled
)

print("\nActual Values:")

```



```

print(y_test)

print("\nPredicted Values:")
print(final_predictions)

print(
    "\nFinal LWR MAE:",
    mean_absolute_error(
        y_test,
        final_predictions
    )
)

```

LWR Results:

	Tau	MAE
0	0.1	4.599975
1	0.3	0.369519
2	0.5	0.104369
3	1.0	0.109501
4	2.0	0.093664

Best Bandwidth:

Tau	2.000000
MAE	0.093664

Name: 4, dtype: float64

Actual Values:

[4.5 4.7]

Predicted Values:

[4.40481157 4.60786066]

Final LWR MAE: 0.09366388424278149

```

import pandas as pd
import numpy as np

```

```

# -----
# Candidate Elimination Algorithm
# -----

```

ANY = "?"

NULL = "∅"

```
def is_consistent(hypothesis, example):
```

```
    for h, x in zip(hypothesis, example):
```

```
        if h != ANY and h != x:
            return False
```

```
    return True
```

```
def more_general_or_equal(h1, h2):
```

```
    for a, b in zip(h1, h2):
```

```
        if a != ANY and a != b:
            return False
```

```
return True
```

```
def generalize_S(S, example):
```

```
    new_S = S.copy()
```

```
    for i in range(len(S)):
```

```
        if S[i] == NULL:
```

```
            new_S[i] = example[i]
```

```
        elif S[i] != example[i]:
```

```
            new_S[i] = ANY
```

```
    return new_S
```

```
def specialize_G(G, example, domains):
```

```
    specializations = []
```

```
    for hypothesis in G:
```

```
        for i in range(len(example)):
```

```
            if hypothesis[i] == ANY:
```

```
                for value in domains[i]:
```

```
                    if value != example[i]:
```

```
                        new_hypothesis = hypothesis.copy()
```

```
                        new_hypothesis[i] = value
```

```
                        specializations.append(
                            new_hypothesis
                        )
```

```
    return specializations
```

```
def candidate_elimination(X, y, domains):
```

```
    n_features = X.shape[1]
```

```
    # Most specific hypothesis
```

```
    S = [NULL] * n_features
```

```
    # Most general hypothesis
```

```
    G = [[ANY] * n_features]
```

```
    for example, label in zip(X, y):
```

```
        example = list(example)
```

```
        # -----
```

```
        # Positive example
```

```
        # -----
```

```
        if label == "Yes":
```

```

        # Remove G hypotheses inconsistent with example
        G = [
            h for h in G
            if is_consistent(h, example)
        ]

        # Generalise S
        S = generalize_S(
            S,
            example
        )

        # Remove overly general hypotheses
        G = [
            h for h in G
            if more_general_or_equal(h, S)
        ]

# -----
# Negative example
# -----

else:

    # Remove S hypotheses that cover
    # the negative example
    if is_consistent(S, example):
        S = S.copy()

    # Specialise G
    new_G = []

    for h in G:

        if is_consistent(h, example):

            specializations = specialize_G(
                [h],
                example,
                domains
            )

            new_G.extend(
                specializations
            )

        else:
            new_G.append(h)

    G = new_G

return S, G

# -----
# Example Dataset
# -----

data = pd.DataFrame({

    "Rainfall": [

```

```

        "Low",
        "High",
        "High",
        "Medium",
        "High",
        "Low"
    ],

    "Temperature": [
        "High",
        "Medium",
        "Low",
        "Medium",
        "Medium",
        "High"
    ],

    "Irrigation": [
        "Low",
        "High",
        "High",
        "Medium",
        "High",
        "Low"
    ],

    "Yield_Class": [
        "No",
        "Yes",
        "Yes",
        "Yes",
        "Yes",
        "No"
    ]
})

# -----
# Separate Features and Target
# -----

X = data[
    ["Rainfall", "Temperature", "Irrigation"]
].values

y = data["Yield_Class"].values

# -----
# Attribute Domains
# -----

domains = [
    ["Low", "Medium", "High"],
    ["Low", "Medium", "High"],
    ["Low", "Medium", "High"]
]

# -----
# Candidate Elimination
# -----

```

```

S, G = candidate_elimination(
    X,
    y,
    domains
)

print("Final Specific Boundary (S):")
print(S)

print("\nFinal General Boundary (G):")

for hypothesis in G:
    print(hypothesis)

```

```

Final Specific Boundary (S):
['?', '?', '?']

```

```

Final General Boundary (G):

```

```

import numpy as np
import pandas as pd
import time

# =====
# Euclidean Distance
# =====

def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

# =====
# Original k-NN From Scratch
# =====

class NaiveKNN:

    def __init__(self, k=5):
        self.k = k

    def fit(self, X, y):
        self.X_train = np.asarray(X, dtype=float)
        self.y_train = np.asarray(y, dtype=float)

    def predict(self, X):

        predictions = []

        for sample in X:

            distances = []

            for i, train_sample in enumerate(
                self.X_train
            ):

                distance = euclidean_distance(
                    sample,
                    train_sample
                )

```

```

        distances.append(
            (distance, self.y_train[i])
        )

    distances.sort(
        key=lambda x: x[0]
    )

    nearest = distances[:self.k]

    prediction = np.mean(
        [value for _, value in nearest]
    )

    predictions.append(prediction)

    return np.array(predictions)

# =====
# Optimised k-NN Using NumPy Vectorisation
# =====

class OptimizedKNN:

    def __init__(self, k=5):
        self.k = k

    def fit(self, X, y):

        self.X_train = np.asarray(
            X,
            dtype=float
        )

        self.y_train = np.asarray(
            y,
            dtype=float
        )

    def predict(self, X):

        X = np.asarray(
            X,
            dtype=float
        )

        predictions = []

        for sample in X:

            # Vectorised distance calculation
            distances = np.sqrt(
                np.sum(
                    (self.X_train - sample) ** 2,
                    axis=1
                )
            )

            # Get indices of k nearest points
            nearest_indices = np.argsort(

```

```

        distances
   )[:self.k]

    prediction = np.mean(
        self.y_train[
            nearest_indices
        ]
    )

    predictions.append(prediction)

    return np.array(predictions)

# =====
# Generate Dataset
# =====

np.random.seed(42)

total_samples = 10000

X = np.random.rand(
    total_samples,
    3
)

y = (
    2 * X[:, 0]
    + 3 * X[:, 1]
    + 1.5 * X[:, 2]
    + np.random.normal(
        0,
        0.1,
        total_samples
    )
)

# =====
# Dataset Sizes
# =====

dataset_sizes = [
    100,
    500,
    1000,
    2000,
    5000
]

results = []

# =====
# Scalability Experiment
# =====

for size in dataset_sizes:

    X_train = X[:size]
    y_train = y[:size]

```

```

X_test = X[size:size + 20]

# -----
# Naive k-NN
# -----

naive_model = NaiveKNN(k=5)

naive_model.fit(
    X_train,
    y_train
)

start = time.perf_counter()

naive_model.predict(
    X_test
)

naive_time = (
    time.perf_counter()
    - start
)

# -----
# Optimised k-NN
# -----

optimized_model = OptimizedKNN(k=5)

optimized_model.fit(
    X_train,
    y_train
)

start = time.perf_counter()

optimized_model.predict(
    X_test
)

optimized_time = (
    time.perf_counter()
    - start
)

# -----
# Speed Improvement
# -----

speedup = (
    naive_time /
    optimized_time
)

results.append({
    "Dataset_Size": size,
    "Naive_Time": naive_time,
    "Optimized_Time": optimized_time
})

```



```

        "Speedup": speedup
    })

# =====
# Display Results
# =====

results_df = pd.DataFrame(
    results
)

print("\nScalability Results:")
print(results_df.to_string(index=False))

```

Scalability Results:

Dataset_Size	Naive_Time	Optimized_Time	Speedup
100	0.025949	0.011823	2.194737
500	0.282198	0.007002	40.301407
1000	0.517339	0.007128	72.574668
2000	0.856874	0.005105	167.866138
5000	1.864304	0.007821	238.365142

```

# =====
# Q7: VISUALISATION, POLICY BRIEF & SDG 2/13
# Google Colab - Complete Code
# =====

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# -----
# 1. CHECK DATA
# -----

print("=" * 60)
print("DATASET INFORMATION")
print("=" * 60)

print("\nShape:")
print(data.shape)

print("\nColumns:")
print(data.columns.tolist())

print("\nFirst 5 rows:")
display(data.head())

# -----
# 2. CLEAN COLUMN NAMES
# -----

data.columns = (
    data.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
    .str.replace("-", "_")
)

```

```
)  
print("\nCleared column names:")  
print(data.columns.tolist())
```

```
# -----  
# 3. AUTOMATICALLY FIND IMPORTANT COLUMNS  
# -----
```

```
def find_column(possible_names, columns):
```

```
    for name in possible_names:  
        if name in columns:  
            return name
```

```
    # Partial matching  
    for col in columns:  
        for name in possible_names:  
            if name in col:  
                return col
```

```
    return None
```

```
# Possible target columns
```

```
target_candidates = [  
    "yield",  
    "crop_yield",  
    "crophyield",  
    "yield_value",  
    "yield_kg",  
    "yield_ton",  
    "production",  
    "crop_production"  
]
```

```
rainfall_candidates = [  
    "rainfall",  
    "rain",  
    "precipitation"  
]
```

```
temperature_candidates = [  
    "temperature",  
    "temp",  
    "avg_temperature",  
    "average_temperature"  
]
```

```
irrigation_candidates = [  
    "irrigation",  
    "irrigation_amount",  
    "water",  
    "water_availability"  
]
```

```
target_col = find_column(  
    target_candidates,  
    data.columns  
)
```

```

rainfall_col = find_column(
    rainfall_candidates,
    data.columns
)

temperature_col = find_column(
    temperature_candidates,
    data.columns
)

irrigation_col = find_column(
    irrigation_candidates,
    data.columns
)

print("\nDetected columns:")
print("Target      :", target_col)
print("Rainfall    :", rainfall_col)
print("Temperature :", temperature_col)
print("Irrigation   :", irrigation_col)

# -----
# 4. IF TARGET WAS NOT FOUND, SELECT NUMERIC COLUMN
# -----

if target_col is None:

    numeric_columns = data.select_dtypes(
        include=np.number
    ).columns.tolist()

    if len(numeric_columns) == 0:

        raise ValueError(
            "No numerical column was found. "
            "Please check your dataset."
        )

    # Use the last numerical column as target
    target_col = numeric_columns[-1]

    print(
        "\nTarget column was not automatically detected."
    )

    print(
        "Using numerical column:",
        target_col
    )

# -----
# 5. CREATE RESULTS FOLDERS
# -----

os.makedirs(
    "results/plots",
    exist_ok=True
)

os.makedirs(

```

```

        "results/tables",
        exist_ok=True
    )

    os.makedirs(
        "results/logs",
        exist_ok=True
    )

# -----
# 6. CONVERT TARGET TO NUMERIC
# -----

data[target_col] = pd.to_numeric(
    data[target_col],
    errors="coerce"
)

# Remove rows where target is missing
plot_data = data.dropna(
    subset=[target_col]
).copy()

# =====
# VISUALISATION 1
# TARGET / CROP YIELD DISTRIBUTION
# =====

plt.figure(figsize=(8, 5))

plt.hist(
    plot_data[target_col],
    bins=15,
    edgecolor="black"
)

plt.xlabel(target_col.replace("_", " ").title())
plt.ylabel("Frequency")
plt.title("Distribution of Target Variable")

plt.tight_layout()

plt.savefig(
    "results/plots/01_target_distribution.png",
    dpi=300
)

plt.show()

# =====
# VISUALISATION 2
# RAINFALL VS TARGET
# =====

if rainfall_col is not None:

    temp_data = plot_data[
        [rainfall_col, target_col]
    ].copy()

```

```

temp_data[rainfall_col] = pd.to_numeric(
    temp_data[rainfall_col],
    errors="coerce"
)

temp_data = temp_data.dropna()

plt.figure(figsize=(8, 5))

plt.scatter(
    temp_data[rainfall_col],
    temp_data[target_col],
    alpha=0.7
)

plt.xlabel(
    rainfall_col.replace("_", " ").title()
)

plt.ylabel(
    target_col.replace("_", " ").title()
)

plt.title(
    "Rainfall vs Target Variable"
)

plt.tight_layout()

plt.savefig(
    "results/plots/02_rainfall_vs_yield.png",
    dpi=300
)

plt.show()

else:

    print(
        "\nRainfall column not found."
    )

# =====
# VISUALISATION 3
# TEMPERATURE VS TARGET
# =====

if temperature_col is not None:

    temp_data = plot_data[
        [temperature_col, target_col]
    ].copy()

    temp_data[temperature_col] = pd.to_numeric(
        temp_data[temperature_col],
        errors="coerce"
    )

    temp_data = temp_data.dropna()

    plt.figure(figsize=(8, 5))

```

```

plt.scatter(
    temp_data[temperature_col],
    temp_data[target_col],
    alpha=0.7
)

plt.xlabel(
    temperature_col.replace("_", " ").title()
)

plt.ylabel(
    target_col.replace("_", " ").title()
)

plt.title(
    "Temperature vs Target Variable"
)

plt.tight_layout()

plt.savefig(
    "results/plots/03_temperature_vs_yield.png",
    dpi=300
)

plt.show()

else:

    print(
        "\nTemperature column not found."
    )

# =====
# VISUALISATION 4
# CORRELATION HEATMAP
# =====

numeric_data = plot_data.select_dtypes(
    include=np.number
)

if numeric_data.shape[1] >= 2:

    correlation = numeric_data.corr()

    plt.figure(figsize=(10, 7))

    plt.imshow(
        correlation,
        aspect="auto"
    )

    plt.colorbar(
        label="Correlation"
    )

    plt.xticks(
        range(len(correlation.columns)),
        correlation.columns,
        rotation=45,

```

```

        ha="right"
    )

    plt.yticks(
        range(len(correlation.columns)),
        correlation.columns
    )

    plt.title(
        "Correlation Matrix"
    )

    plt.tight_layout()

    plt.savefig(
        "results/plots/04_correlation_matrix.png",
        dpi=300
    )

    plt.show()
else:

    print(
        "\nNot enough numerical columns for correlation."
    )

# =====
# VISUALISATION 5
# ACTUAL VS PREDICTED
# =====

# Check whether predictions from Q2/Q3 already exist
prediction_available = False

if "predictions" in globals():

    try:

        y_true = np.asarray(
            y_test,
            dtype=float
        )

        y_pred = np.asarray(
            predictions,
            dtype=float
        )

        if len(y_true) == len(y_pred):

            prediction_available = True

    except:

        prediction_available = False

# Try final_predictions from Q3
if not prediction_available:

```

```

if "final_predictions" in globals():

    try:

        y_true = np.asarray(
            y_test,
            dtype=float
        )

        y_pred = np.asarray(
            final_predictions,
            dtype=float
        )

        if len(y_true) == len(y_pred):

            prediction_available = True

    except:

        prediction_available = False

if prediction_available:

    plt.figure(figsize=(8, 5))

    plt.scatter(
        y_true,
        y_pred,
        alpha=0.7
    )

    min_value = min(
        y_true.min(),
        y_pred.min()
    )

    max_value = max(
        y_true.max(),
        y_pred.max()
    )

    plt.plot(
        [min_value, max_value],
        [min_value, max_value]
    )

    plt.xlabel("Actual Values")
    plt.ylabel("Predicted Values")

    plt.title(
        "Actual vs Predicted Values"
    )

    plt.tight_layout()

    plt.savefig(
        "results/plots/05_actual_vs_predicted.png",
        dpi=300
    )

```



```

plt.show()

else:

    print(
        "\nPredictions from Q2/Q3 were not found."
    )

    print(
        "Run your Q2 or Q3 model first "
        "to generate this graph."
    )

# =====
# VISUALISATION 6
# SCALABILITY ANALYSIS
# =====

# Check if scalability results from Q5 exist

if "results_df" in globals():

    scalability_df = results_df.copy()

    print(
        "\nResults dataframe found:"
    )

    print(
        scalability_df
    )

    required_columns = [
        "Dataset_Size",
        "Naive_Time",
        "Optimized_Time"
    ]

    if all(
        col in scalability_df.columns
        for col in required_columns
    ):

        plt.figure(figsize=(8, 5))

        plt.plot(
            scalability_df["Dataset_Size"],
            scalability_df["Naive_Time"],
            marker="o",
            label="Naive k-NN"
        )

        plt.plot(
            scalability_df["Dataset_Size"],
            scalability_df["Optimized_Time"],
            marker="o",
            label="Optimized k-NN"
        )

        plt.xlabel(
            "Dataset Size"
        )

```

```

        plt.ylabel(
            "Execution Time (seconds)"
        )

        plt.title(
            "k-NN Scalability Analysis"
        )

        plt.legend()

        plt.tight_layout()

        plt.savefig(
            "results/plots/06_scalability.png",
            dpi=300
        )

        plt.show()

        # Save table
        scalability_df.to_csv(
            "results/tables/scalability_results.csv",
            index=False
        )

    else:

        print(
            "\nScalability dataframe does not "
            "contain the required columns."
        )

else:

    print(
        "\nQ5 scalability results not found."
    )

    print(
        "Run Q5 first to generate the scalability graph."
    )

# =====
# SAVE BASIC STATISTICS
# =====

statistics = plot_data.describe()

statistics.to_csv(
    "results/tables/descriptive_statistics.csv"
)

print("\nDescriptive Statistics:")
display(statistics)

# =====
# FINAL SUMMARY
# =====

print("\n" + "-" * 60)

```

```
print("Q7 VISUALISATION COMPLETE")
print("=" * 60)

print("\nGenerated files:")

for root, dirs, files in os.walk(
    "results/plots"
):
    for file in files:
        print(
            os.path.join(
                root,
                file
            )
        )

print("\nResults folder:")
print("results/")
print(" └─ plots/")
print(" └─ tables/")
print(" └─ logs/")
```


=====

DATASET INFORMATION

=====

Shape:
(6, 4)

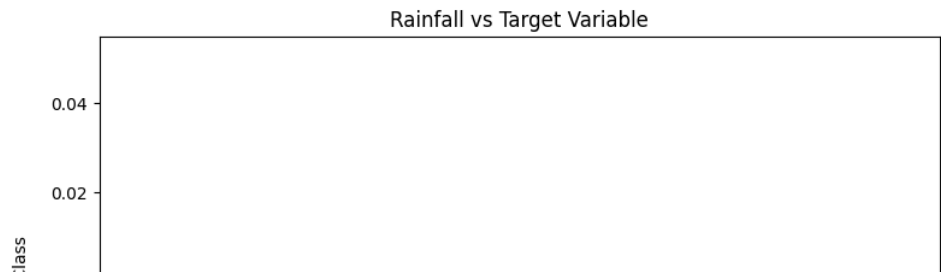
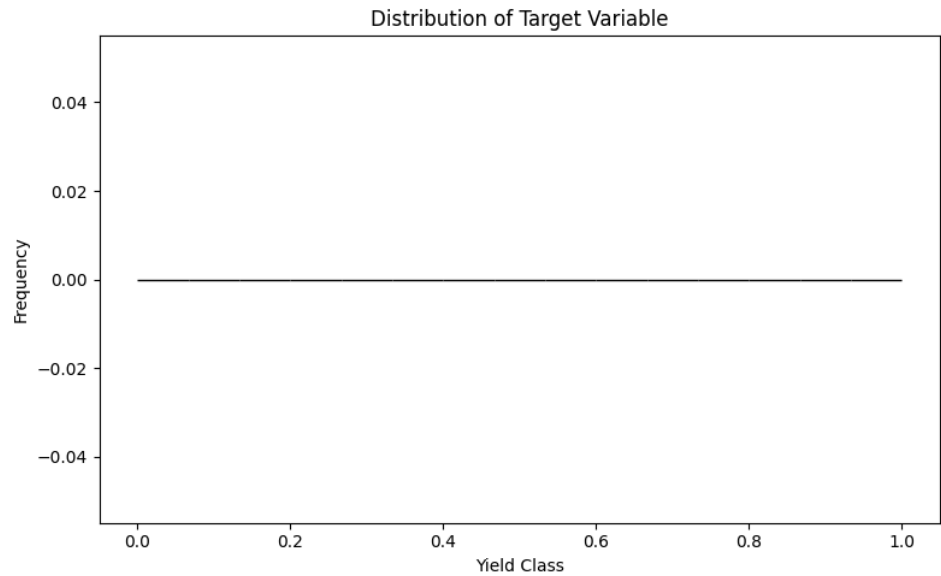
Columns:
['Rainfall', 'Temperature', 'Irrigation', 'Yield_Class']

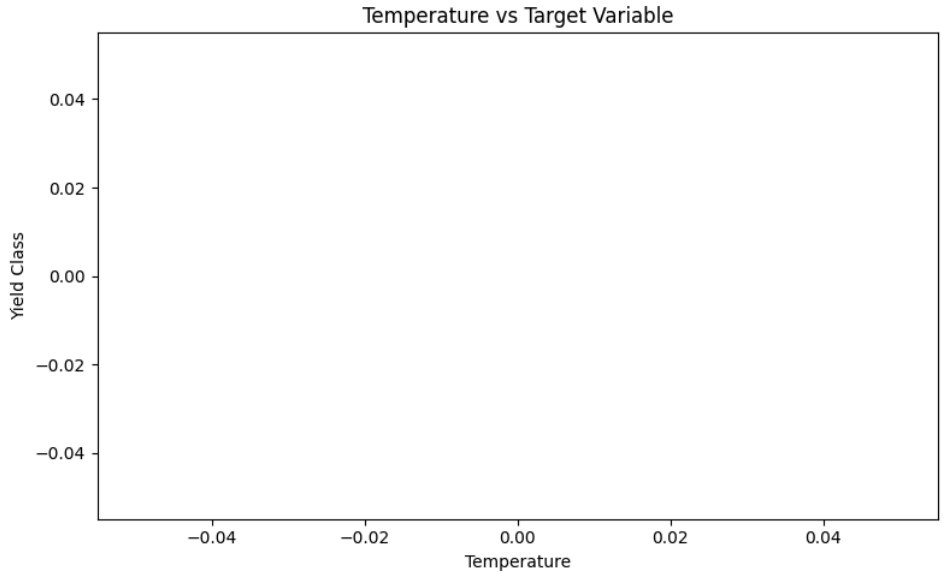
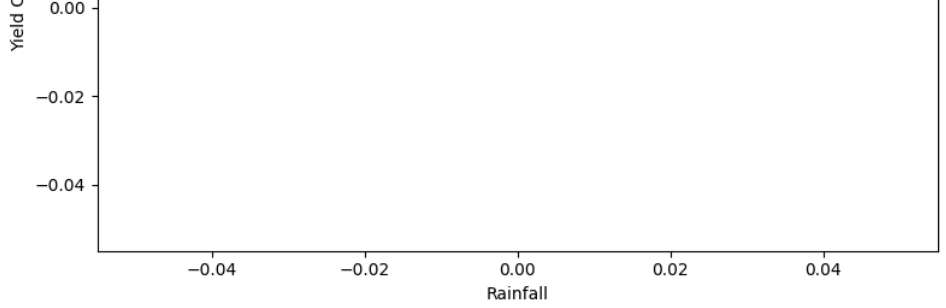
First 5 rows:

	Rainfall	Temperature	Irrigation	Yield_Class	
0	Low	High	Low	No	
1	High	Medium	High	Yes	
2	High	Low	High	Yes	
3	Medium	Medium	Medium	Yes	
4	High	Medium	High	Yes	

Cleaned column names:
['rainfall', 'temperature', 'irrigation', 'yield_class']

Detected columns:
Target : yield_class
Rainfall : rainfall
Temperature : temperature
Irrigation : irrigation





Not enough numerical columns for correlation.

